

Vitamin Gelb — Beyond the List (PostFinance)

Technische Informationen für die Jury

Aktueller Stand des Sourcecodes

GitHub: <https://github.com/matthauslukas77-debug/vitamingelb-beyond-the-list> · Branch `main`

Live: <https://www.vitamingelb.ch>

Selber starten: `cd Code && npm install && npm run dev`

Das läuft ohne Schlüssel und ohne Backend. `npm test` und `npm run build` sind grün.

Ausgangslage

Worauf haben wir uns fokussiert?

Die Challenge stellt drei Fragen: Was ist regelmässig? Was ist ungewöhnlich? Was verändert sich?

Wir beantworten alle drei. Das meiste Gewicht liegt auf den letzten beiden, weil es dafür heute gar keine Antwort gibt. Wir haben acht Leute gefragt, wie sie herausfinden, ob ein Monat normal war. Eine Antwort lautete: «Gar nicht.»

Zielgruppe: Privatpersonen zwischen 19 und 30. Das sind die Leute, mit denen wir gesprochen haben. KMU haben wir weggelassen, weil wir kein einziges KMU-Gespräch geführt haben.

Wo die Funktionen sitzen: nicht in einem neuen Dashboard. Unsere Gesprächspartner öffnen die App für den Saldo und schliessen sie wieder. Die bestehenden Analysen kennt fast niemand. Wer darauf mit noch einer Kachel antwortet, macht denselben Fehler nochmal. Unsere Sachen sitzen auf dem Startbildschirm, im Suchfeld und in einem Hinweis, der von selbst kommt.

Weggelassen: Multibanking, weil PostFinance das seit November 2025 hat und unsere Zielgruppe mit zwei Konten nichts davon hat. Und ein Chat-Fenster, aus demselben Grund wie oben.

Welche technischen Grundsatzentscheide haben wir gefällt?

1. Nachbau und eigene Schicht getrennt. `src/app/` ist die heutige App, nachgebaut. `src/insights/` ist unser Beitrag. Verbunden sind sie über benannte Einhängpunkte. Nimmt man unsere Schicht weg, steht der Ist-Zustand da. Nur so kann man vorher und nachher zeigen.

2. Erfundene Daten, einmal erzeugt. PostFinance stellt keinen Datensatz und keine Schnittstelle bereit. Unsere vier Personas liegen als TypeScript im Repository, 670 bis 1147

Buchungen über 24 Monate. Jede Demo läuft gleich, und es gibt keine Datenschutzfrage.

3. Alle Beträge in ganzen Rappen. Kein Fließkomma. Gerundet wird erst bei der Anzeige.

4. Keine UI- und keine Diagrammbibliothek. Jede Fläche und jede Kurve ist von Hand geschrieben. Ein zugekauftes Diagramm bringt seine eigenen Farben mit, und dann passt der Ton neben dem Nachbau nicht mehr.

5. Erkennen im Code, formulieren im Modell. Das ist der wichtigste Entscheid. Warum, steht unter «Implementation».

6. Alles Wichtige läuft ohne Schlüssel. Supabase und Apertus schalten sich ab, wenn nichts konfiguriert ist. Die App funktioniert trotzdem.

Technischer Aufbau

Welche Komponenten und Frameworks haben wir verwendet?

Was	Womit	Wofür
Oberfläche	React 19, TypeScript 5.9 (strict)	Beide Schichten sind React-Komponenten
Build	Vite 7	Entwicklung und Produktionsbundle
Tests	Vitest 3	491 Tests in 31 Dateien, alle grün
Gestaltung	CSS mit Custom Properties	290 Tokens aus dem PostFinance-Design-System, inklusive Dark Mode. Keine UI-Bibliothek
Diagramme	SVG von Hand	Verlauf, Donut, Prognose, Budget-Blasen
Fachlogik	eigenes TypeScript	Abo-Erkennung, Buchungstext zerlegen, Prognose, Budget
Daten	feste TypeScript-Dateien	vier Personas, mit einem Python-Generator einmalig erzeugt
Lokal speichern	localStorage	Einstellungen und Budget überleben ein Neuladen, ohne Login
Backend (optional)	Supabase: Postgres und zwei Edge Functions (Deno)	wird erst beim ersten Zugriff nachgeladen
Sprachmodell	Apertus (Swiss AI)	formuliert Sätze und ordnet Fragen zu. Rechnet nie
Auslieferung	Vercel	vitamingelb.ch, als Web-App installierbar

Laufzeitabhängigkeiten sind React, React-DOM und, nur wenn konfiguriert, der Supabase-Client. Sonst nichts.

Wozu und wie werden diese eingesetzt?

Am Anfang steht ein fester Datensatz pro Persona: Konten, Buchungen im Textformat des echten Kontoauszugs, Daueraufträge, pendente Aufträge.

Daraus rechnet reines TypeScript ohne Oberfläche:

- `domain/recurring.ts` erkennt Zahlungsreihen aus Text, Betrag und Abstand. Nicht aus einer

Kennung im Datensatz, denn die gibt es in echten Kontodaten nicht.

- `domain/booking.ts` zerlegt den Buchungstext in Zahlungsart, Kaufdatum, Karte, Händler, Land.
- `insights/engine/balance.ts` baut den Kontostandverlauf und rechnet ihn mit den erkannten

Reihen in die Zukunft weiter.

Sichtbar wird das über Einhängpunkte. Die Nachbau-Bildschirme rendern an bestimmten Stellen `<Slot name="home.accountRow" fallback={...} />`. Ist dort eine Komponente von uns eingetragen, ersetzt sie den Baustein des Nachbaus. Sonst zeigt die App genau den Ist-Zustand. So lässt sich jede Funktion einzeln an- und abschalten.

Das Sprachmodell kommt zuletzt, wenn alles gerechnet ist. Die Edge Function bekommt fertige Zahlen, nie Buchungen. Der Prompt entsteht dort, der Schlüssel liegt dort.

Implementation

Gibt es etwas Spezielles, was ihr erwähnen wollt?

Wir haben den PostFinance-Budgetrechner vermessen, nicht nachempfunden. Unser Budget vergleicht gegen einen Richtwert. Damit «du zahlst mehr fürs Wohnen als ein vergleichbarer Haushalt» stimmt, muss der Richtwert im selben Massstab stehen wie das Original. Wir haben Struktur und Rechenlogik des öffentlichen Rechners rekonstruiert und gegen 2513 Messpunkte seiner eigenen Schnittstelle geprüft. Ergebnis: höchstens 1 Franken Abweichung pro Jahr, die Klassenzuordnung stimmt in allen Fällen. Alles nachlesbar in `Misc/PF-Budgetrechner/`.

Sparen ist keine Ausgabe. Jede Buchung bekommt eine von fünf Richtungen: raus, rein, verschoben, abgerechnet, ausgelegt. Eine Umbuchung aufs eigene Sparkonto ist verschoben, die Kartenabrechnung ist abgerechnet. Sonst zählt man doppelt. Zwei Leute haben uns unabhängig voneinander fast denselben Satz gesagt: «Ich tue 500 Franken auf das Sparkonto, dann ist das wie eine Ausgabe, obwohl es gar nicht weg ist.»

Eine Antwort pro Händler, nicht pro Buchung. Wer eine Kategorie korrigiert, korrigiert sie für alle Buchungen dieses Händlers, auch für künftige. Eine Person hat 20 Einkäufe beim selben Laden. Heute sind das 20 Handgriffe, bei uns einer.

Tests, die auch das Aussehen prüfen. Uns ist ein Fehler bis auf die Live-Seite gerutscht: Beim Umbau blieb die halbe CSS-Datei doppelt stehen, und die alte Kopie überschrieb die neue. Alle

Blasen waren rot. Weder die Typprüfung noch die Tests konnten das sehen, weil CSS für beide unsichtbar war. Seither liest eine Testdatei die Stilvorlagen als Text und prüft Dinge, die sonst niemand prüft. Jede dieser Regeln steht dort, weil sie einmal verletzt wurde.

Was ist aus technischer Sicht besonders cool an eurer Lösung?

Dass wir Apertus einsetzen, ist nicht das Interessante. Interessant ist, wo wir es nicht tun.

Wir haben zuerst gemessen. Beide Apertus-Größen gegen 90 Testtransaktionen mit drei eingebauten Signalen:

- Beide addieren 14 Beträge falsch. Jedes Mal.
- Auf «Was ist ungewöhnlich?» nennen beide den Lohn. Also das Regelmässigste, was es gibt.
- Mit einer fertig gerechneten Zahl formulieren beide sauber und erfinden nichts dazu.

Daraus folgt die Aufteilung:

Stelle	Modell	Aufgabe
Erkennung	keines	Reihen, Abweichungen, Budget, Prognose
explain	Apertus 70B	einen Satz aus fertigen Zahlen formulieren
ask	Apertus 8B	eine Frage auf eines von sechs Werkzeugen abbilden

Der 8B ist der Router, weil nur er saubere Tool-Calls liefert und in 0,3 Sekunden antwortet. Der 70B schreibt das bessere Deutsch. Beides gemessen.

Vier Kontrollen, und jede steht wegen eines echten Vorfalls da:

- **Zahlenwache.** Jede Zahl im Antworttext muss im Prompt gestanden haben. Sie hat gleich beim

ersten Lauf etwas gefunden: Der 70B schrieb «fast CHF 400», wo die Differenz 349 war.

- **Argumentwache.** Dasselbe für Namen. Auf «Kannst du mir diesen Namen auflösen?» antwortete der

8B mit UBS, im nächsten Lauf mit Amazon. Beide stehen nirgends in der Frage. Ein Verbot im Systemtext half nicht, die Prüfung schon.

- **Themenzaun.** Zu Anlagen, Vorsorge, Hypotheken und Urteilen über die Person sagt der Assistent

nichts. Die Prüfung läuft auf dem Server, weil der Endpunkt auch ohne unseren Browser erreichbar ist.

- **Der Katalog ist die Grenze.** ask gibt nie Text zurück, den das Modell geschrieben hat. Es

kommt ein Werkzeugname heraus oder nichts. Eine erfundene Zahl kann diesen Weg nicht nehmen, weil auf ihm keine Zahlen fließen.

Unser liebster Befund: Der 8B wählt fast immer das richtige Werkzeug und verpackt es fast nie richtig. Von 16 korrekten Wahlen kamen nur 7 als sauberer Tool-Call zurück. Neun standen als

blosser Text in der Antwort, einer davon mit einem durchgesickerten Token des Chat-Templates. Wer nur die Tool-Calls liest, wirft mehr als die Hälfte weg. Unser Leser nimmt beide Formen und prüft dafür streng gegen den Katalog: 20 von 21 statt 11 von 21, bei unverändertem Modell.

Und das Modell ist verzichtbar. Die Absichtserkennung läuft zuerst im Code. Gegen 149 Formulierungen aus unseren Interviews trägt sie 62 Prozent, ohne Netz und ohne Schlüssel. Fällt Apertus aus, verliert die App Sprachverständnis, aber keine Funktion.

Abgrenzung / Offene Punkte

Welche Abgrenzungen haben wir bewusst vorgenommen? Weshalb?

Kein Schreibpfad in die Datenbank. Supabase ist angebunden, das Schema liegt als Migration im Repository. Genutzt werden nur die beiden Edge Functions. Keine Tabelle wird gelesen oder geschrieben. Der Grund ist eine Zusage an diese Jury: `npm install && npm run dev` läuft ohne einen einzigen Schlüssel. Der Preis ist, dass in der Demo Angelegtes ein Neuladen nicht übersteht.

Zahlungen laufen durch, buchen aber nicht. Der Zahlungsfluss ist vollständig bedienbar und legt Aufträge an, erzeugt aber keine Buchung. Eine echte Buchung würde jede abgeleitete Zahl verändern: Kontostand, Prognose, Budget, Signale. Dafür bräuchte es eine einzige Quelle für Buchungen. Heute lesen 30 Stellen in 23 Dateien die Persona-Daten direkt. Das in der verbleibenden Zeit umzubauen hätte zwei Wahrheiten riskiert. Eine ehrliche Attrappe ist uns lieber als eine halb echte Rechnung.

Kein Chat-Fenster, obwohl wir einen Assistenten haben. Er sitzt im Suchfeld: eine Frage, eine Antwort, kein Verlauf. Ein Chat-Bildschirm wäre wieder ein Ort, den man aufsuchen muss, und genau daran sind die Analysen gescheitert. Der Preis ist benannt: «Wer ist Hornbach?» wird beantwortet, «und letztes Jahr?» nicht.

Kein Rate-Limit auf der Edge Function. Begrenzt sind heute nur Länge, Antwortlänge und Zeit. Für einen Prototyp mit erfundenen Daten tragbar, für den Betrieb nicht.

Alle Daten sind erfunden. Vier Personas, 24 Monate, aus einem festen Startwert erzeugt. Kein Bankanschluss, keine echten Kontodaten.

Was als Nächstes käme

1. **Eine Quelle für Buchungen.** Erst absichern: Mit leerer eigener Schicht muss jede abgeleitete

Zahl gleich bleiben wie heute. Dann erst die Verhaltensänderung.

1. **Danach bucht die Zahlung wirklich**, und ein Sparhinweis verschwindet, weil die Erkennung die

Umbuchung sieht. Nicht auf Knopfdruck.

1. **Dann Persistenz.** Eigene Buchungen gehören in eine eigene Tabelle, nicht als Einfügung in

die Demo-Daten.

1. **Rate-Limit und Härtung der Leseregeln.** Die Policies für Gespräche, Ziele und Pläne stehen

auf «für alle lesbar». Solange dort nur erfundene Daten liegen, ist das tragbar. Sobald jemand eigene Texte eingibt, gehört die Regel an die Sitzung gebunden.